DistriBook: Distributed Systems Implementation Report

Distributed Classroom and Laboratory Booking System

Final-Year B.Tech Computer Engineering | Subject: Distributed Systems

Author: Pradnya | Repository: `DS_FA`

> **Verification note.** This report is written from the project description and the locally reported test results. Items marked **[Verify in code]** must be confirmed against the source files (exact file names, function names, routes, payload fields) before submission. Nothing in this document claims a feature that was not stated as implemented.

1. Abstract and Problem Statement

Educational institutions share a limited pool of classrooms and laboratories. Bookings must respect the academic timetable, must never overlap, and often need faculty and administrative approval. When this is handled by one monolithic application, a failure or change in one area affects everything, and concurrent requests can produce double bookings.

DistriBook addresses this with a microservice architecture. Four independent backend services (Resource, Timetable, Booking, Notification) communicate through synchronous REST/HTTP and asynchronous RabbitMQ messaging. PostgreSQL provides persistence, Docker runs the infrastructure, and a React frontend presents one unified application.

The project demonstrates the following Distributed Systems concepts in a working system: distributed architecture, middleware, message-oriented communication, Lamport logical clocks, database-backed concurrency control, containerization and service-health monitoring. It also states honestly which syllabus topics are **not** implemented.

Problem statement: Design and implement a distributed booking system in which independent services cooperate to prevent conflicting reservations, deliver event notifications asynchronously, and expose a logical ordering of events.

2. Objectives and Scope

2.1 Objectives

- Manage classroom and laboratory resources through independent services.
- Provide timetable-aware booking and prevent conflicting reservations.
- Demonstrate synchronous communication through REST APIs.
- Demonstrate asynchronous communication through RabbitMQ.
- Maintain a logical ordering of booking events using Lamport clocks.
- Prevent conflicting concurrent bookings through database transactions and row locking.
- Monitor the availability of individual services.

2.2 In Scope

Four backend services, a React frontend, PostgreSQL storage, RabbitMQ messaging, Docker-based infrastructure, mock identity-based authorization, Lamport clocks in the Booking and Notification services, and health endpoints on all backend services.

2.3 Out of Scope

Production-grade authentication, cloud deployment, vector clocks, leader election, distributed deadlock detection, distributed file systems, serverless execution and blockchain. These are discussed in Section 12 as syllabus topics that are not implemented.

3. System Requirements and Technology Stack

3.1 Technology Stack

Layer	Technology	Purpose
Frontend	React with Vite	User interface, dev server on port 5173
Backend	Node.js (TypeScript) **[Verify in code]**	Four independent services
Database	PostgreSQL	Persistent storage

Layer	Technology	Purpose
ORM	Prisma	Service-specific schemas and queries
Messaging	RabbitMQ	Asynchronous booking events
Containers	Docker and Docker Compose	PostgreSQL and RabbitMQ runtime
Algorithms	Lamport logical clock	Event ordering
Concurrency	DB transactions with row locking	Conflict prevention

3.2 Service Ports

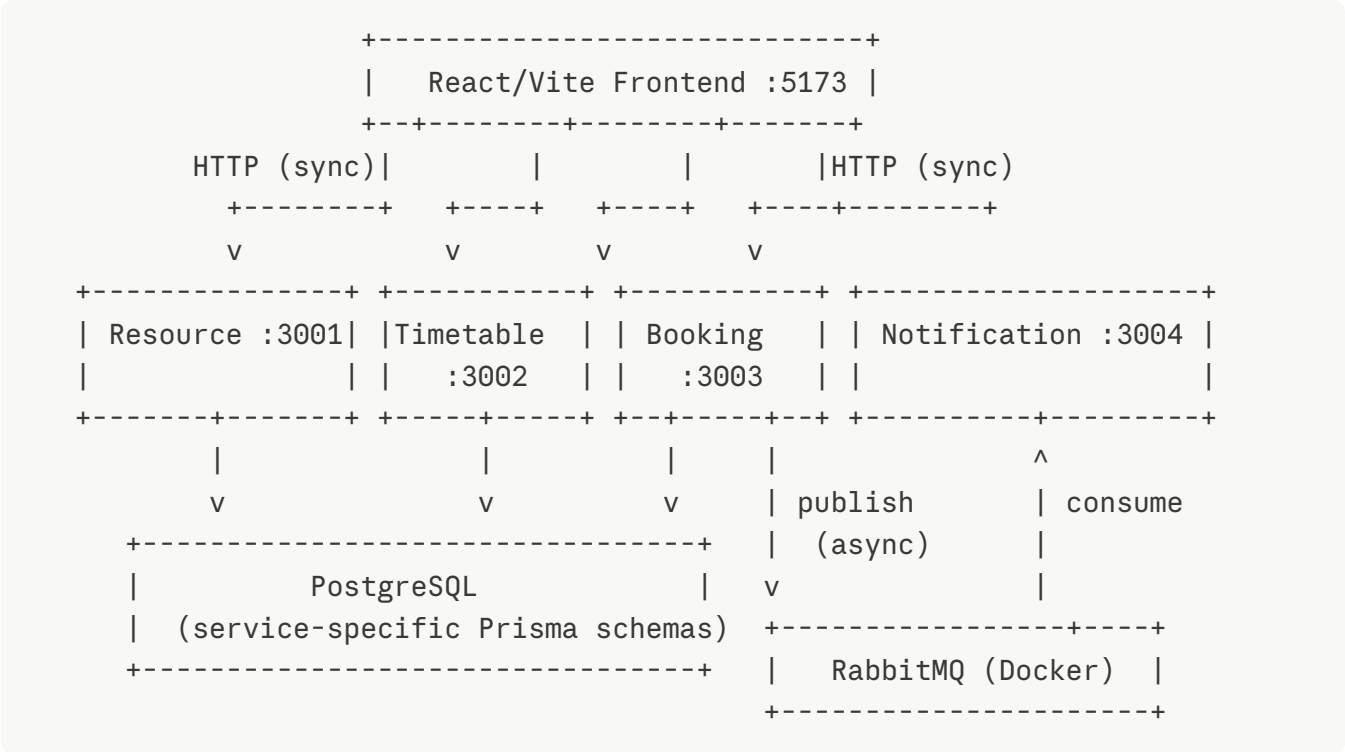
Component	Port
Resource Service	3001
Timetable Service	3002
Booking Service	3003
Notification Service	3004
Frontend (Vite)	5173

3.3 Requirements to Run

- Node.js and npm.
- Docker Desktop (for PostgreSQL and RabbitMQ).
- PowerShell on Windows for `system_check.ps1`.
- Environment variables for database and broker connections **[Verify in code: list names from `.env` / `docker-compose.yml`, never include secret values]**.

4. Architecture and Service Responsibilities

4.1 Architecture Overview



Synchronous paths: Frontend to each service over HTTP. Asynchronous path: Booking Service to RabbitMQ to Notification Service. Which inter-service HTTP calls exist (for example, whether the Booking Service calls the Timetable or Resource Service) must be confirmed from the code **[Verify in code]**.

4.2 Resource Service (port 3001)

Manages classrooms, laboratories and other bookable resources. Provides retrieval and authorized resource-management operations.

4.3 Timetable Service (port 3002)

Manages academic timetable data. Provides timetable retrieval and import, and supplies the scheduled-class information used to detect booking conflicts.

4.4 Booking Service (port 3003)

Handles booking creation, retrieval, cancellation and approval workflows. Validates requests against existing reservations, academic schedules and other constraints. Uses a database transaction with row locking to stop conflicting concurrent bookings. Publishes booking events to RabbitMQ and maintains an in-memory Lamport clock.

4.5 Notification Service (port 3004)

Consumes booking events from RabbitMQ, applies the Lamport receive rule, and persists notifications for users. Notification titles can display the logical timestamp, for example `[L=62]`.

4.6 Frontend

React/Vite application with Dashboard, Resources, Timetable, Bookings, Approvals, Notifications and Services pages, plus the Distributed Systems documentation page.

4.7 Identity and Authorization

The system uses a mock identity mechanism: the frontend sends an `X-User-Email` header, and services apply role-based permissions. Tests cover missing and invalid identities and role permissions. This is suitable only for development and demonstration; it is **not** production-grade authentication (see Section 13).

5. Database and Message-Broker Design

5.1 Database

PostgreSQL runs in Docker. Each service uses its own Prisma schema, so each service owns its data and no service reads another service's tables directly **[Verify in code: confirm whether services use separate databases or separate schemas in one database]**.

Service	Data owned (typical)	Verify
Resource	Resources	Model names and fields
Timetable	Timetable entries	Model names and fields
Booking	Bookings, approval state	Status values, overlap fields

Service	Data owned (typical)	Verify
Notification	Notifications	Recipient, title, body, read flag

5.2 RabbitMQ Design

RabbitMQ is the message-oriented middleware. Complete this table from the publisher and consumer source:

Item	Value
Exchange name and type	**[Verify in code]**
Routing keys	**[Verify in code]**
Queue(s) and durability	**[Verify in code]**
Message persistence	**[Verify in code]**
Consumer acknowledgement mode	**[Verify in code]**
Retry / dead-letter handling	**[Verify in code]**
Payload fields	**[Verify in code]**
Lamport header name	**[Verify in code]**

The Lamport timestamp travels in both the message payload and the message headers.

5.3 Why Message-Oriented Communication Here

Creating a notification is not part of deciding whether a booking is valid. Publishing an event decouples the two: the booking request returns without waiting for notification processing, and the Notification Service can be slower or briefly unavailable without blocking bookings.

6. API Communication Patterns

6.1 Synchronous: REST over HTTP

The caller sends an HTTP request and waits for the response. Typical operations: retrieve resources, access timetables, create and approve bookings, retrieve notifications. REST here is request-response over HTTP; it is not described as RPC because no dedicated RPC framework (gRPC, JSON-RPC and so on) has been established.

6.2 Asynchronous: RabbitMQ

The Booking Service publishes and continues. The Notification Service consumes later. Neither knows the other's address; both know only the broker.

6.3 Verified Route Inventory

Fill from the route files. Only the following are stated as existing:

Service	Route	Purpose
All four	`GET /health`	Public health check
Booking	`GET /lamport`	Current Lamport clock value
Notification	`GET /lamport`	Current Lamport clock value
Others	**[Verify in code]**	Resources, timetable, bookings, approvals, notifications

7. End-to-End Workflows

7.1 Booking Request (synchronous)

```
User -> Frontend -> POST booking (Booking :3003)
  1. Identify user (X-User-Email) and check role
  2. Begin database transaction
  3. Lock the relevant row(s)
  4. Check overlap with existing bookings
  5. Check timetable / maintenance conflicts
  6. If conflict: roll back, return HTTP 409
  7. If valid: insert booking, commit, return HTTP 201
  8. Publish booking event (async, see 7.2)
```

7.2 Booking Event and Notification (asynchronous)

```
Booking Service                RabbitMQ              Notification Service
   |  L = L + 1                    |                          |
   |-- publish(event, L) --------->|                          |
   |                               |-- deliver -------------->|
   |                               |      L = max(L_local, L_msg) + 1
   |                               |      persist notification (title may
show [L=n])
   |                               |<-- ack (if configured) --|
```

7.3 Approval Flow

A booking can progress through approval stages (the reported flow includes faculty and administrative approval **[Verify in code: exact roles and status transitions]**). Each state change can publish an event, so a single booking produces a causally ordered sequence of events, which is what the Lamport test exercises.

8. Lamport Logical Clocks

8.1 Simple Explanation

Physical clocks on different machines never agree perfectly, so timestamps from "wall time" cannot reliably say which of two events happened first. A Lamport clock sidesteps this. Each service keeps a counter. Every time it sends a message it bumps the counter

and attaches it. Every time it receives a message it jumps its counter ahead of both its own value and the message's value. The result: if one event could have caused another, it always has a smaller number.

8.2 Technical Definition

Each process keeps a scalar counter L, initially 0.

Sender rule:

`L = L + 1`, then attach L to the outgoing message.

Receiver rule:

```
L = max(L_local, L_message) + 1
```

Clock condition (the guarantee):

If event a happened-before event b (a → b), then L(a) < L(b).

What it does not guarantee: The converse. L(a) < L(b) does not prove a → b. Two concurrent events can have any ordering of timestamps. Detecting concurrency precisely requires vector clocks, which are not implemented.

8.3 Where It Lives in DistriBook

Question	Answer
Where is the clock stored?	In-memory counter in the Booking Service and in the Notification Service
When does it increment?	On publishing a booking event (sender); on consuming one (receiver)
How does the message carry it?	In the payload and in a message header **[Verify in code: header name]**
How does the consumer update?	Reads the incoming value and applies `max(local, incoming) + 1`
How is it displayed?	Notification titles can carry a prefix such as `[L=62]`; `GET /lamport` returns the current value
Legacy message with no header?	Incoming value treated as 0, so the receiver still increments its own clock

[Verify in code] whether any additional local events (not only send and receive) also increment the counter, and cite the file and function.

8.4 Worked Example

Notification Service local clock is 61. It receives a booking event stamped 13.

`L_new = max(61, 13) + 1 = 62`

The receiver was already ahead of the sender, and the rule still produces a valid, larger timestamp. The notification is labelled `[L=62]`.

A second example with a causal chain:

Step	Event	Process	Clock before	Message L	Clock after
1	Booking created, event sent	Booking	4	n/a	5
2	Event received	Notification	2	5	6
3	Approval recorded, event sent	Booking	5	n/a	6
4	Event received	Notification	6	6	7

Timestamps rise along the causal chain: 5 < 6 < 6 (send) < 7. Note the Notification clock jumps to 7 at step 4 because `max(6, 6) + 1 = 7`.

8.5 Pseudocode

```
state: L = 0                        # per service, in memory

function send(event):
    L = L + 1
    message = { payload: event, lamport: L }
    publish(message, headers = { lamport: L })

function onReceive(message):
    Lm = header(message).lamport OR 0     # legacy messages count as 0
    L = max(L, Lm) + 1
    persistNotification(message, title = "[L=" + L + "] ...")
```

8.6 Causal Ordering of Booking Events

Booking created, then faculty approval, then administrative approval is a causal chain: each event is triggered after the previous one has been observed by the system. Lamport timestamps increase along this chain. The Lamport test checks monotonic timestamps across a multi-event sequence.

Logical timestamps do **not** provide exactly-once delivery or guarantee every event is processed.

8.7 Limitations

- Counters are in memory and reset to 0 when a service restarts, so timestamps are not unique or monotonic across restarts.
- Timestamps can jump ahead of the local event count; gaps are normal behaviour, not an error.
- With multiple replicas, concurrent events can receive equal timestamps. A total order needs a tie-break such as the pair (L, process ID).
- Lamport clocks measure logical order, not physical time, and do not synchronize physical clocks.
- They cannot tell whether two events are concurrent (see vector clocks).
- Durable clock persistence is not implemented.

9. Concurrency Control and Conflict Prevention

9.1 The Problem

Two users submit a booking for the same room and time at the same instant. Without control, both checks see "free" and both inserts succeed: a double booking (a race condition).

9.2 The Mechanism

The Booking Service performs the check-and-insert inside a **database transaction** with **row locking**, plus an **overlap check** against existing bookings and timetable or maintenance conflicts:

```
BEGIN transaction
  lock resource row (blocks competing transactions on the same resource)
  check overlap with existing bookings
  check timetable / maintenance conflicts
  if conflict -> ROLLBACK, respond 409
  else        -> INSERT booking, COMMIT, respond 201
```

The second concurrent request waits for the lock, then sees the first booking and is rejected.

9.3 Verified Result

The concurrency test submitted two overlapping requests for the same resource and time. One returned HTTP 201 and the other HTTP 409.

9.4 Important Distinction

This is **database-backed concurrency control**. It is not Lamport's distributed mutual exclusion algorithm, and it should not be presented as one. It relies on a single shared PostgreSQL instance as the coordinator, which is also a single point of failure and a scaling limit.

10. Health Monitoring and Failure Limitations

Every backend service exposes a public `GET /health`. The frontend Services page polls these endpoints and shows status, response time and uptime where available.

10.1 What It Provides

Detection of an unresponsive service from the client's point of view.

10.2 What It Does Not Provide

- Automatic failover or restart.
- Durable event replay: if a message is lost or the consumer is down, health polling does not recover it.
- Distinguishing a crashed service from a slow network.
- A global-state snapshot (such as Chandy-Lamport). It reports independent per-service availability only.

- A formal beacon protocol.

Health checks are therefore **failure detection**, not **fault tolerance**.

10.3 Delivery and Consistency Considerations

- **Dual write:** a booking is committed to PostgreSQL and then an event is published. If the publish fails after the commit, the notification is lost. The standard remedy is the transactional outbox pattern (future scope).
- **Delivery semantics:** whether delivery is at-most-once or at-least-once depends on acknowledgement and durability settings **[Verify in code]**. With at-least-once delivery the consumer should be idempotent to avoid duplicate notifications.
- **Eventual consistency:** booking state and notification state live in separate services and are consistent only eventually.

11. Syllabus Mapping

Status key: **Implemented**, **Partial**, **Not implemented**.

Unit I: Introduction

Topic	Status	Evidence in DistriBook
Definition and goals	Implemented	Four independent services behind one UI; modularity, resource sharing
Distributed architecture	Implemented	Microservices, separate ports, separate data ownership
Design issues	Partial	Separation of concerns and sync/async split; scalability not demonstrated
Middleware	Implemented	RabbitMQ as message-oriented middleware
Overview / model of distributed computations	Implemented	Booking, publish, receive, notify as events ordered by Lamport clocks
Virtualization	Partial	Docker containers (OS-level, not full VMs)

Topic	Status	Evidence in DistriBook
Distributed multimedia	Not implemented	No media or streaming feature

Unit II: Communication

Topic	Status	Evidence
Communication fundamentals	Implemented	HTTP request-response plus asynchronous broker messaging
RPC	Not implemented	REST/HTTP only; no dedicated RPC protocol shown
Message-oriented communication	Implemented	Booking Service to RabbitMQ to Notification Service
Stream-oriented communication	Not implemented	No continuous stream
Peer-to-peer messaging	Not implemented	Client/server and broker only
WebRTC	Not implemented	Not used
Names, identifiers, addresses	Partial	Service URLs and ports, record IDs; no name service or discovery
Fault tolerance	Partial	Health detection only; no failover or replay

Unit III: Synchronization

Topic	Status	Evidence
Clock synchronization (physical)	Not implemented	Lamport clocks are logical, not physical
Logical clocks / Lamport's algorithm	Implemented	In-memory counters in Booking and Notification; `/lamport` endpoints

Topic	Status	Evidence
Global state	Not implemented	Health page is not a snapshot algorithm
Vector algorithm	Not implemented	Scalar clocks only
Election algorithms	Not implemented	No leader election
Mutual exclusion	Partial	DB transaction and row locking prevent double booking; not a distributed mutual-exclusion algorithm
Beacon protocol	Not implemented	Health polling is not a formal beacon protocol
Lodha-Kshemkalyani fair mutual exclusion	Not implemented	Theory only
Knapp's deadlock-detection classification	Not implemented	Theory only

Unit IV: Emerging Paradigms

Topic	Status	Evidence
Distributed web-based systems	Implemented	React frontend, HTTP APIs, broker, database
Distributed object-based systems	Not implemented	No distributed-object framework
Distributed file systems	Not implemented	PostgreSQL is not a DFS
Serverless	Not implemented	Containerized services
Blockchain / ledgers	Not implemented	Not used

Topic	Status	Evidence
Distributed-database trade-offs	Partial	Single PostgreSQL; no replication or consensus; discussed theoretically
Megaport, Cloudflare, AWS, Hadoop, Kubernetes	Not implemented	Syllabus case studies, not project features

12. Testing

12.1 Commands

```
node backend/test_extended_authorization.js
node backend/test_concurrency.js
node backend/test_lamport.js
powershell -ExecutionPolicy Bypass -File .\system_check.ps1
```

Run them with the services, PostgreSQL and RabbitMQ up.

12.2 Reported Local Results

Test	Reported result	Re-run and attach output
Extended authorization and flow	11 passed, 0 failed (identity, roles, booking, overlap rejection, timetable conflict, notification persistence)	Date and raw output **[Add]**
Concurrency	One HTTP 201 and one HTTP 409 for two overlapping requests	Date and raw output **[Add]**
Lamport	Four scenarios passed: send/receive rules, monotonic multi-event sequence, legacy message without header, advance after higher timestamp	Date and raw output **[Add]**
System check	HTTP 200 from all four `/health` endpoints and tested business APIs	Date and raw output **[Add]**

These are local results only. They do not prove cloud availability or behaviour under every failure.

12.3 Manual Demonstration Checklist

1. Start Docker, then the four services and the frontend. Open the Services page; all four should show healthy.
2. Create a booking; confirm it appears under Bookings.
3. Create an overlapping booking for the same resource and time; expect a conflict response.
4. Open Notifications; confirm a notification with an `[L=n]` prefix.
5. Call `GET /lamport` on Booking and Notification; confirm values advance after further bookings.
6. Stop one service; confirm the Services page marks it down, and note that no automatic recovery happens.

13. Limitations, Security and Future Scope

13.1 Limitations

- Lamport counters are in memory and lost on restart.
- No multi-replica ordering strategy.
- No vector clocks, election, global-state snapshots or distributed deadlock detection.
- Single PostgreSQL instance and single RabbitMQ broker: single points of failure.
- Health monitoring detects but does not recover.
- Possible event loss between database commit and broker publish.

13.2 Security Considerations

- The `X-User-Email` header is trivially forgeable; any client can claim any identity.
- A public deployment needs real authentication (for example signed tokens), server-side authorization on every route, TLS, secret management, input validation and rate limiting.
- Do not commit credentials; keep `.env` out of version control and out of this report.
- Restrict RabbitMQ management and database ports from public access.

13.3 Future Scope

- Persist Lamport clocks or adopt hybrid or vector clocks.
- Transactional outbox, durable queues, retries and a dead-letter queue.
- Idempotent consumers with message IDs.
- Real authentication and an API gateway.
- Cloud deployment (hosted PostgreSQL and RabbitMQ), planned work only until verified.
- Replication, circuit breakers and automatic restarts for fault tolerance.

14. Viva Questions and Model Answers

Q1. Why is DistriBook a distributed system? It consists of independent services (Resource, Timetable, Booking, Notification) running as separate processes, communicating only by messages over a network (HTTP and RabbitMQ), and presenting one unified application.

Q2. What is the role of RabbitMQ? It is message-oriented middleware. The Booking Service publishes events and the Notification Service consumes them, decoupling the two in time and in knowledge of each other.

Q3. Is your REST communication RPC? No. It is HTTP request-response. RPC would present a remote operation as a local procedure call through a dedicated framework, which this project does not use.

Q4. State Lamport's rules. Send: `L = L + 1` and attach L. Receive: `L = max(L_local, L_msg) + 1`.

Q5. What does Lamport guarantee? If a happened-before b, then L(a) < L(b). The converse is false.

Q6. Why is it not enough to compare timestamps to know causality? A scalar timestamp cannot distinguish "caused" from "concurrent". Vector clocks can; they are not implemented.

Q7. Do Lamport clocks synchronize clocks? No. They order events logically and say nothing about physical time.

Q8. Why can the Notification clock be larger than the Booking clock? It also counts its own receive events and takes the maximum with incoming values; being ahead is normal.

Q9. What happens to the clock on restart? It resets to 0 because it is in memory. Persistence is future work.

Q10. How do you prevent double booking? A database transaction with row locking and an overlap check. The second concurrent request waits, sees the first booking and is rejected with HTTP 409.

Q11. Is that mutual exclusion? It is database-backed concurrency control with a central coordinator (PostgreSQL), not a distributed mutual-exclusion algorithm such as Lamport's or Ricart-Agrawala.

Q12. Is your health page a global snapshot? No. It polls each service independently. A snapshot algorithm (Chandy-Lamport) also records channel states.

Q13. Does the system tolerate failures? It detects service unavailability; it does not fail over, replay lost events or replicate state, so it is not fault tolerant in the full sense.

Q14. What if the broker publish fails after the booking commits? The notification would be lost. The fix is the outbox pattern: write the event to a table in the same transaction and publish from there.

Q15. Why is the mock identity unsafe? Any client can set `X-User-Email` to any value. Production needs verified tokens and server-side authorization.

Q16. Which syllabus topics are not implemented? Vector clocks, election algorithms, global-state snapshots, beacon protocol, fair mutual exclusion, distributed deadlock detection, RPC, streaming, P2P, WebRTC, distributed objects, DFS, serverless and blockchain.

15. Source References

Complete this table from the repository so every claim has a citation.

Concept	File	Function / route
Lamport clock (Booking)	**[Add path]**	**[Add]**
Lamport clock (Notification)	**[Add path]**	**[Add]**
RabbitMQ publisher	**[Add path]**	**[Add]**
RabbitMQ consumer	**[Add path]**	**[Add]**

Concept	File	Function / route
Booking transaction and lock	**[Add path]**	**[Add]**
Health endpoints	**[Add path]**	`GET /health`
Frontend Services page	**[Add path]**	**[Add]**
Docker Compose	`docker-compose.yml`	Services, ports, environment
Tests	`backend/test_*.js`, `system_check.ps1`	See Section 12

16. Conclusion

DistriBook applies Distributed Systems principles in a working system: independent microservices, REST and RabbitMQ communication, PostgreSQL persistence, Docker containers, Lamport logical clocks, transactional concurrency control and service-health monitoring. Its strongest direct syllabus coverage is distributed architecture, middleware, message-oriented communication, Lamport's algorithm, concurrency control and web-based distributed systems. Several topics remain theoretical and are stated as such, which keeps the report accurate and defensible in a viva.